

交通大数据

Python数据分析应用



Python数据分析应用

□ 学习目标

- 了解Python的特点、优势及编程环境配置
- 理解Python的基础语法，如数据类型、基本操作、容器等
- 掌握Python的流程控制，函数与类的定义与调用，及异常处理方法



Python数据分析应用

□ 学习目标

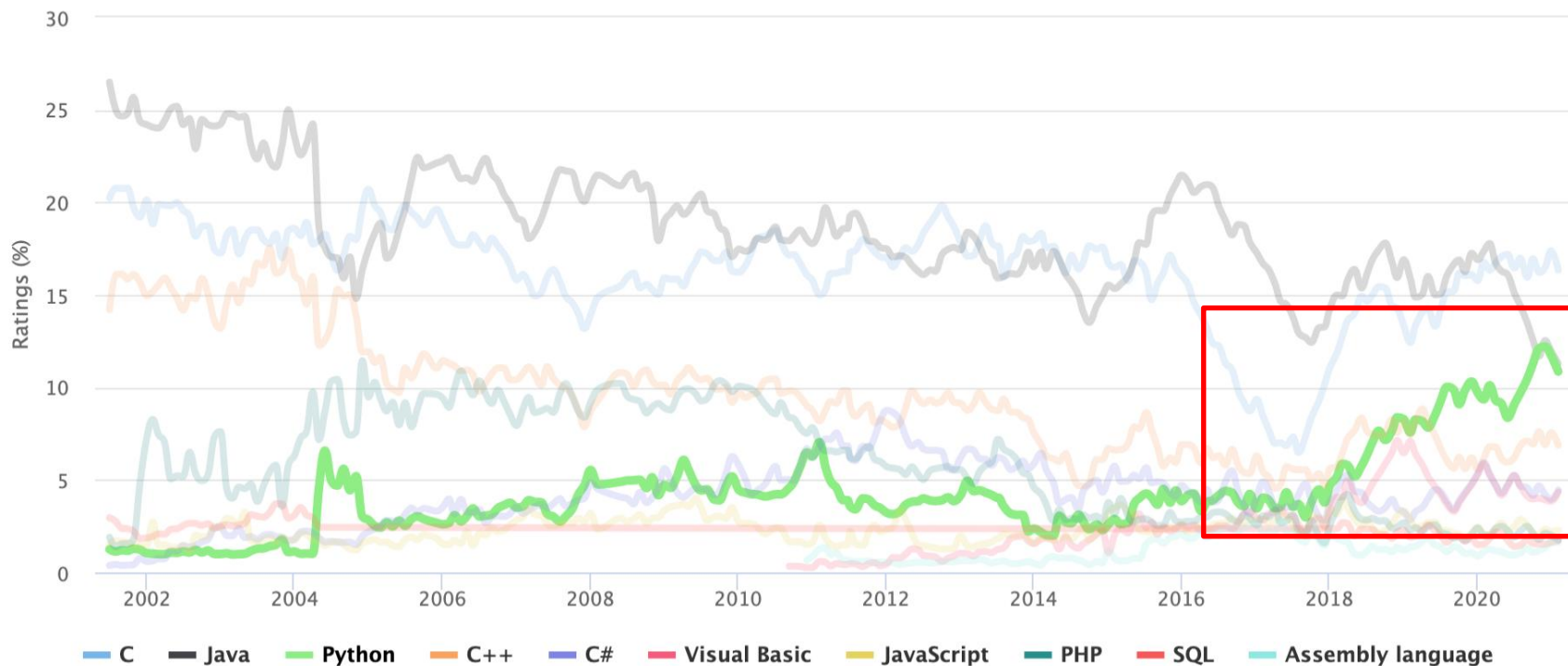
- **了解Python的特点、优势及编程环境配置**
- 理解Python的基础语法，如数据类型、基本操作、容器等
- 掌握Python的流程控制，函数与类的定义与调用，及异常处理方法



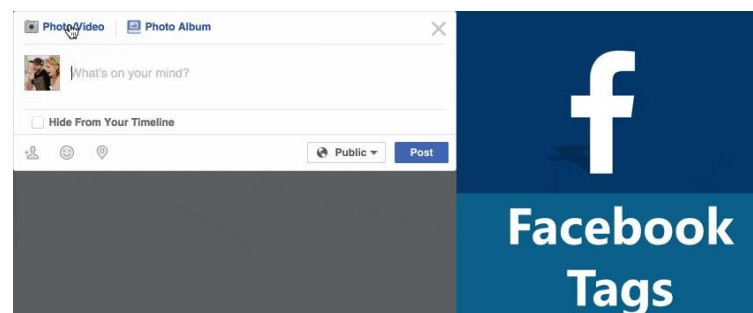
为什么使用Python?

TIOBE Programming Community Index

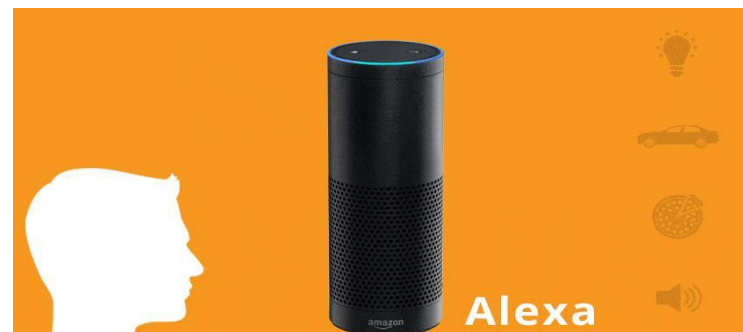
Source: www.tiobe.com



为什么使用Python?

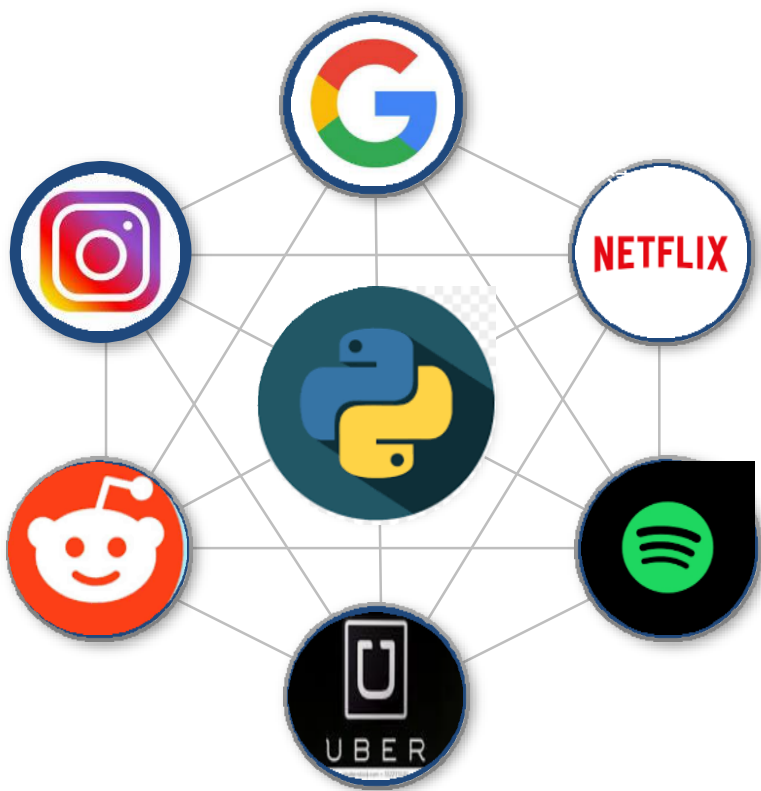


给图像和帖子打标签



语音识别、图像检测

为什么使用Python?



豆瓣douban

“用python开发的网站开始公测”

2005年3月6日下午,我弱弱地给中国Python用户组的邮件列表发了一封email。题目是“用python开发的网站开始公测”:

欢迎来到<https://douban.com>使用豆瓣的读书者网络和书价比较。

豆瓣全部使用python开发。网站后台的搜索引擎基于Twisted, GUI基于Quixote。还用到了cElementTree, MySQLdb和PIL。除了这些豆瓣基本是从头开发(一个人四、五个月)完成。

如果你有爱读书或爱写书评的朋友,也欢迎推荐豆瓣给他们。最好的方式是通过豆瓣的邀请功能。

谢谢!



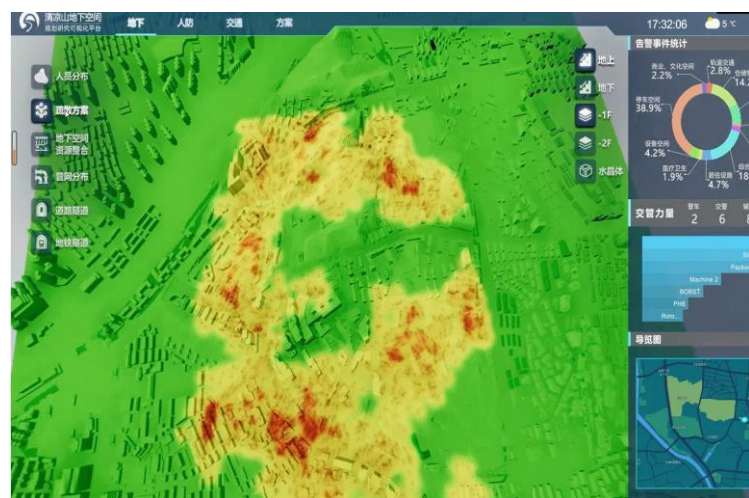
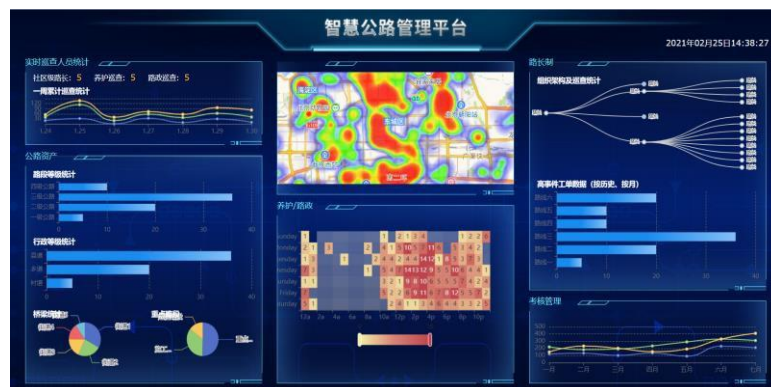
为什么使用Python?



为什么使用Python?

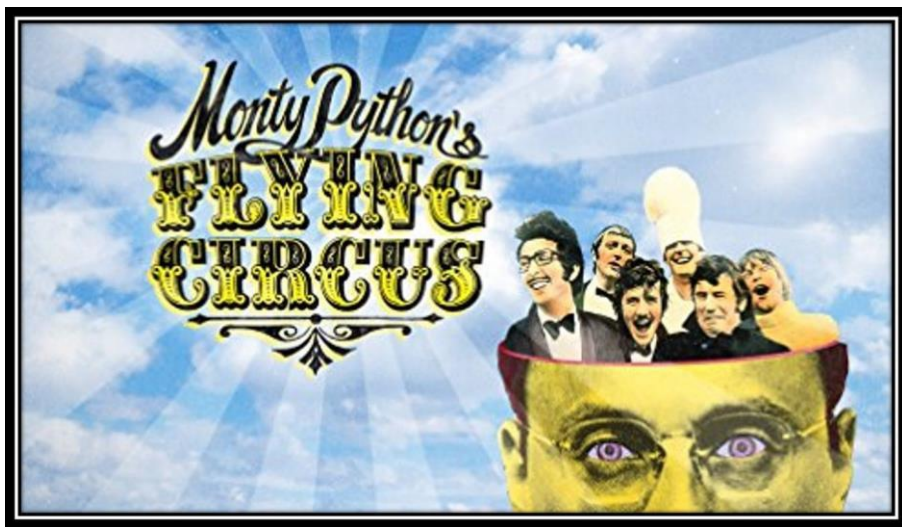
□ Python与交通

- 入门简单，提高交通分析效率
- 兼容性强，能够和其他语言的代码一起解决交通问题
- 社区环境好，资源丰富，有大量教程和完善的工具包，提供多种交通数据接口



Python概述

□ Python的起源



由英国BBC喜剧系列
“Monty Python 's Flying Circus” 命名



Guido van Rossum

荷兰程序员，于1989年底
始创了Python，1991年初，
Python发布了第一个公开
发行版

Python概述

□ Python的特点

**Python is powerful... and fast;
plays well with others;
runs everywhere;
is friendly & easy to learn;
is Open.**

“优雅、明确、简单、易读”

- 无需编译，执行简单
- 独立于平台，可移植性好
- 文本文件，便于调试与修改
- 代码简洁，语法简单，可读性高
- 拥有有庞大且成熟的第三方模块
- 开源，社群庞大，快速解决代码问题
-



Python VS C++

□ 解释型语言 VS 编译型语言

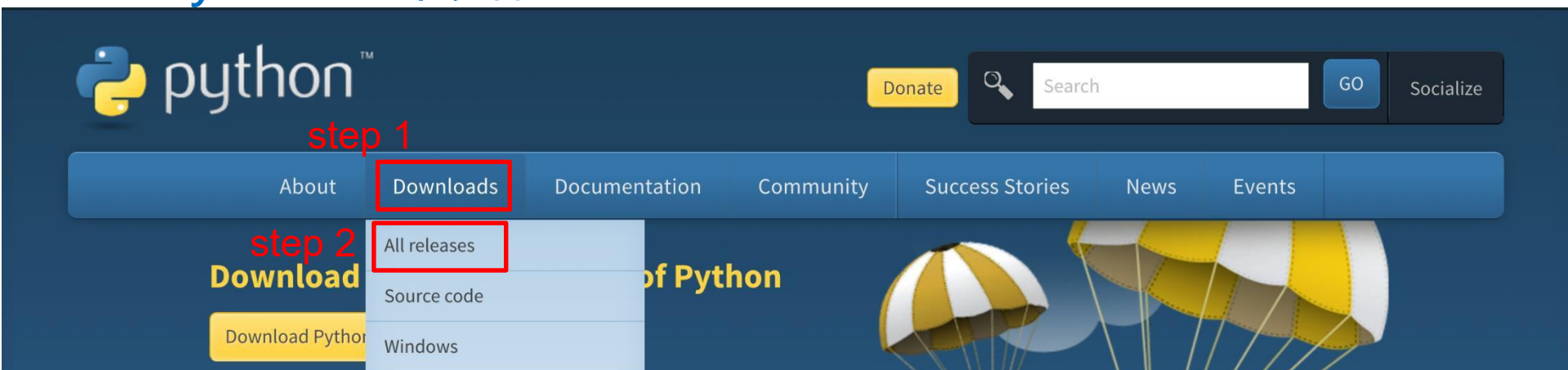


Python作为解释型语言的优势：

- 无需编译，执行更简单；
- 独立于平台，可移植性更好；
- 可读性高，便于调试与修改；

Python概述

□ Python的环境配置



<https://www.python.org/downloads/release/python-392/>

□ 提示

- 如果使用Mac, 已安装Homebrew, 使用终端命令brew install python3安装;
- 如果使用windows, 安装时需勾选Add Python 3.9 to PATH

[详见课本2.1.1节](#)



Python概述

❑ 使用Anaconda进行软件包管理与部署



Individual
Edition

Team
Edition

Enterprise
Edition

Anaconda.org



Our **Terms of Service** have recently changed.

Using Anaconda for commercial purposes?
View our products.

Anaconda Installers

Contains over 200 curated packages that are securely built, highly optimized, and tested together to ensure compatibility.

[View All Installers](#)

Anaconda Packages

Install more packages with **conda**.
Anaconda, Inc. maintains 1,000+ professionally built packages for data science.

[View All Packages](#)

Miniconda Installers

The minimal way to bootstrap conda onto a system. Contains only conda and its dependencies.

[View All Installers](#)

<https://repo.anaconda.com/archive/>



Python概述

□ Spyder

快速切换工作路径



详细使用见参考资料：spyder教程



Python概述

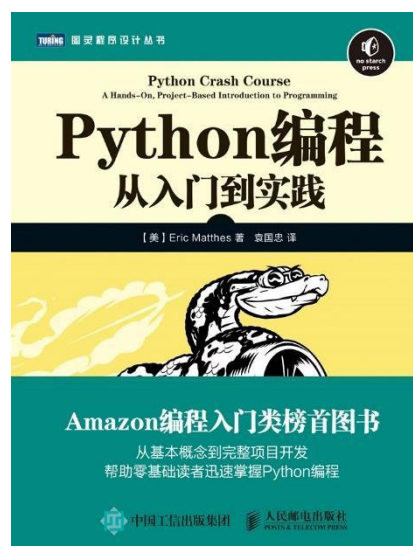
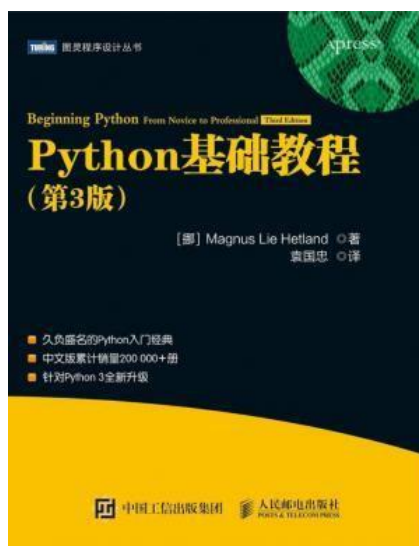
□ 推荐学习资料

- 廖雪峰的python教程

<https://www.liaoxuefeng.com/wiki/1016959663602400>

- 阅读文档 <https://www.runoob.com/python3/python3-tutorial.html>、

- 阅读书籍：建议选择一本快速阅读，用实践夯实基础



Python数据分析应用

□ 学习目标

- 了解Python的特点、优势及编程环境配置
- **理解Python的基础语法，如数据类型、基本操作、容器等**
- 掌握Python的流程控制，函数与类的定义与调用，及异常处理方法



最佳学习方法：基于数据，动手实践



Hello world!

Python基础语法

无需声明预处理器指令和命名空间

C++

```
#include <iostream>  
using namespace std;  
int main(){  
    cout << "Hello World!" << endl;  
    return 0;  
}
```

使用print()输出

不以分号结尾

Python

```
print("Hello, World!")
```



Python基础语法

□ 数据类型

- 整数值 `print(1)`
- 浮点型 `print(0.1)`
- 字符串型 `print("good")`
- 布尔值 `print(True)`
- 空值 `print(None)`

2min 试一试

- `0.1+0.2==0.3`的结果是True还是False?
- 输入 `"First" + 0.1` 是否报错?
- 运行`print( 'I' m a good student )`
是否报错?



Python基础语法

□ 数据类型

试一试

- $0.1+0.2==0.3$ 的结果是True还是False?
- 输入 `"First" + 0.1` 是否报错?
- 运行`print( 'I' m a good student )`是否报错?

```
In [1]: 0.1+0.2==0.3
```

```
Out[1]: False
```

```
In [2]: 'First'+0.1
```

```
File "<ipython-input-2-454cb2b8a751>", line 1  
    'First'+0.1  
          ^
```

```
SyntaxError: invalid character in identifier
```

```
In [3]: print('I'm a good student')
```

```
File "<ipython-input-3-a1d7e6f3f1e6>", line 1  
    print('I'm a good student')  
          ^
```

```
SyntaxError: invalid syntax
```



Python基础语法

□ 数据类型

试一试

- $0.1+0.2==0.3$ 的结果是True还是False?
- 输入 `"First" + 0.1` 是否报错?
- 运行`print( 'I' m a good student )`是否报错?

- 使用`round()`进行四舍五入

```
In [1]: round(0.1)+round(0.2)==round(0.3)
Out[1]: True
```

- 使用数据类型转换函数`str()`

```
In [2]: "First"+str(0.1)
Out[2]: 'First0.1'
```

- 单双引号交替使用或转义字符(`\`)

```
In [3]: print("I'm a good student")
I'm a good student
```

```
In [4]: print('I\'m a good student')
I'm a good student
```



Python基础语法

□ 基本运算

命名用字母、数字和下划线组成，不能数字开头

- 数学运算 $+$, $-$, $*$, $/$, $\%$, $//$, $**$
- 字符串运算符 $+$, $*$, $[i]$, $[m:n]$, in , $not\ in$
- **变量**的定义无需说明数据类型（动态变量），常量一般大写，如PI
- 字符串常见处理函数 $len()$, $lower()$, $strip()$, $count()$, $join()$

- 计算2的平方用 2^2 还是 $2**2$?

`s=" transport BIG DATA"`

提示：阅读课本2.2.2节

- 返回s的长度;
- 将字符串转换为大写字母;
- 统计字符串中a出现的次数;



Python基础语法

□ 基本运算

试一试

- 计算2的平方用 2^2 还是 $2**2$?

`s=" transport BIG DATA"`

- 返回s的长度;
- 将字符串转换为大写字母;
- 统计字符串中a出现的次数

```
In [1]: 2^2
```

```
Out[1]: 0
```

```
In [2]: 2**2
```

```
Out[2]: 4
```

```
In [3]: s = "transport BIG DATA"
```

```
In [4]: len(s)
```

```
Out[4]: 18
```

```
In [5]: s.upper()
```

```
Out[5]: 'TRANSPORT BIG DATA'
```

```
In [6]: s.count('a')
```

```
Out[6]: 1
```



Python基础语法

□ 缩进与注释

1min 试一试

在代码编辑区，对当前代码：

- 使用#进行单行注释
- 调整缩进，查看是否报错

Spyder中Ctrl+4/5:块注释/撤销块注释

```
#s = "transport BIG DATA"
# =====
# s = "transport BIG DATA"
# s = "transport BIG DATA"
# s = "transport BIG DATA"
# =====
    print("s=2")
^
IndentationError: unexpected indent
```

注释冗余，实际程序编写时，注释非常重要，但务必言之有物

~~#请输入你的名字~~
yourname = input("请输入你的名字: ")



Python中的容器

□ 列表(list) —————→ 提示：阅读课本2.3.1节

使用 “[]” 定义

3min 试一试

```
s = ['a1','a2','a3','a4']
```

- 切片操作,取出a2和a3
- 将a4添加到末尾
- 将a5插入到指定位置
- 删除末尾元素

有序存储

元素可删改

Python中的容器

□ 列表(list)

试一试

`s = ['a1','a2','a3','a4']`

- 切片操作, `s[1:3]`
- 将a4添加到末尾, `s.append('a4')`
- 将a5插入到指定位置, `s.insert(1,'a5')`
- 删除末尾元素: `s.pop()`

使用 “[]” 定义

有序存储

元素可删改

```
In [1]: s=['a1','a2','a3','a4']
```

```
In [2]: s[1:3]
```

```
Out[2]: ['a2', 'a3']
```

```
In [3]: s.append('a4')
```

```
In [4]: s
```

```
Out[4]: ['a1', 'a2', 'a3', 'a4', 'a4']
```

```
In [5]: s.insert(1,'a5')
```

```
In [6]: s
```

```
Out[6]: ['a1', 'a5', 'a2', 'a3', 'a4', 'a4']
```

```
In [7]: s.pop()
```

```
Out[7]: 'a4'
```

```
In [8]: s
```

```
Out[8]: ['a1', 'a5', 'a2', 'a3', 'a4']
```



Python中的容器

□ 元组(tuple) ———> 提示：阅读课本2.3.2节

1min 试一试

s = ('a1','a2','a3','a4')

- 如何定义只有一个元素的元组?

使用 “()” 定义

元素不可删改

有序存储

```
In [1]: s=('a1')
```

```
In [2]: s  
Out[2]: 'a1'
```

```
In [3]: s=('a1',)
```

```
In [4]: s  
Out[4]: ('a1',)
```

注意加逗号



Python中的容器

□ 字典(dict) ———→ 提示：阅读课本2.3.3节

使用 “{}” 定义

3min 试一试

s = {'a1':118.733,'a2':118.788}

- 查询a2的值
- 增加a3=118.781
- 取出字典所有键
- 取出字典所有值
- 取出字典所有项（键-值对）

使用键-值进行无序存储

元素可删改



Python中的容器

□ 字典(dict)

试一试

```
s = {'a1':118.733,'a2':118.788}
```

- 查询, s.get('a2')
- 增加a3, s['a3']=118.781
- 取出字典的键, s.keys()
- 取出字典的值, s.values()
- 取出字典的项, s.items()

```
In [1]: s = {'a1':118.733,'a2':118.788}

In [2]: s.get('a2')
Out[2]: 118.788

In [3]: s['a3']=118.781

In [4]: s
Out[4]: {'a1': 118.733, 'a2': 118.788, 'a3': 118.781}

In [5]: s.keys()
Out[5]: dict_keys(['a1', 'a2', 'a3'])

In [6]: s.values()
Out[6]: dict_values([118.733, 118.788, 118.781])

In [7]: s.items()
Out[7]: dict_items([('a1', 118.733), ('a2', 118.788), ('a3', 118.781)])
```



Python中的容器

□ 集合(set) ———→ 提示：阅读课本2.3.4节

使用列表定义

3min 试一试

s = **set**([1,2,2,3]) #输出为{1,2,3}

- 增加元素4
- 删除元素3
- 定义集合p由3,4,5组成
- 对s和p取交集、并集和差集

不重复，不存储键

元素可删改



Python中的容器

□ 集合(set)

试一试

```
s = set([1,2,2,3])
```

- 增加元素, s.add(4)
- 删除元素, s.remove(3)
- 定义集合p由3,4,5组成
- 使用&, |, -, 对s和p取交集、并集和差集

```
In [1]: s = set([1,2,2,3])
```

```
In [2]: s.add(4)
```

```
In [3]: s.remove(3)
```

```
In [4]: s  
Out[4]: {1, 2, 4}
```

```
In [5]: p = set([3,4,5])
```

```
In [6]: s & p  
Out[6]: {4}
```

```
In [7]: s | p  
Out[7]: {1, 2, 3, 4, 5}
```

```
In [8]: s - p  
Out[8]: {1, 2}
```



Python中的容器

□ 小结

- 容器：可包含其他对象的对象，包含序列、映射和集合等
- 序列：**列表**和**元组**，其中**列表可修改**，一般能满足对序列的需求；
- 字典：**字典**是唯一内置映射类型，值存储在**键下**，**不按照顺序排列**；
- 集合：**集合**可变，因此不能作为键，由内置类set实现，**元素不可重复**。



Python数据分析应用

□ 学习目标

- 了解Python的特点、优势及编程环境配置
- 理解Python的基础语法，如数据类型、基本操作、容器等
- **掌握Python的流程控制，函数与类的定义与调用，及异常处理方法**



流程控制语句

□ 条件判断语句

单条件

```
if 条件:  
    执行语句1  
else:  
    执行语句
```

注意冒号

多条件

```
if 条件:  
    执行语句1  
if 条件:  
    执行语句2  
else:  
    执行语句  
if 条件1:  
    执行语句1  
elif 条件2:  
    执行语句2  
else:  
    执行语句
```



流程控制语句

□ 条件判断语句

3min 试一试

if 条件1: 执行语句1

elif 条件2:

 执行语句2 else:

 执行语句

- 使用上述语句，根据速度，打印

```
v = 10
if v < 60:
    print('当前低于最低限速，请加速')
elif v > 120:
    print('当前高于最高限速，请减速')
else:
    print('请保持当前车速')
```

速度	低于60	[60-120]	高于120
提示	当前低于最低限速，请加速	请保持当前速度	当前高于最高限速，请减速



流程控制语句

□ 循环语句

- while 循环

循环判断语句 注意冒号

while True:
 执行语句

break: 提前停止循环, 直接运行下一块代码
continue: 跳过当前循环, 进入下一次循环

- for 循环

for i in range(1,10) :

 print(i)

左闭右开

注意缩进



流程控制语句

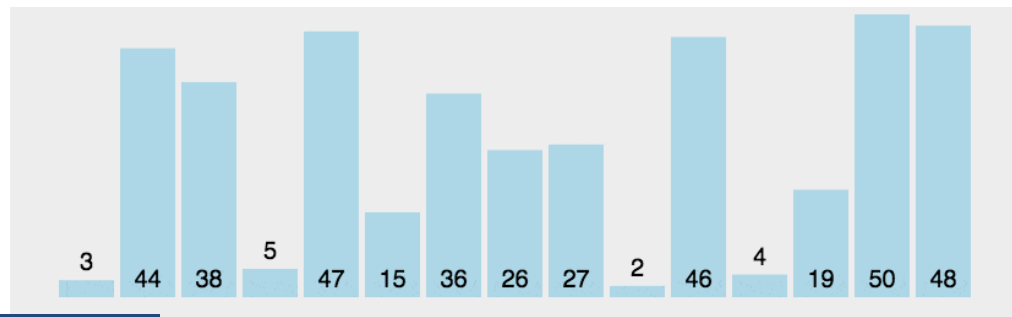
□ 循环语句

冒泡排序-C++

```
#include <iostream>
using namespace std;

void bubblesort(int *a, int n){
    int tmp;
    int flag;
    for (int i=0;i<n;i++){
        flag = 1;
        for (int j=0;j<n-i-1;j++){
            if (a[j]>a[j+1]){
                tmp = a[j+1];
                a[j+1] = a[j];
                a[j] = tmp;
                flag = 0;
            }
        }
        if (flag) break;
    }
}

int main(){
    int a[] = {8,6,7,5,1,3,4,2};
    int l = sizeof(a)/sizeof(int);
    bubblesort(a, l);
    for (int i=0;i<l;i++){
        cout << a[i] << " ";
    }
    cout << endl;
    return 0;
}
```



15min 试一试

使用Python循环语句对8, 6, 7, 5, 1, 3, 4, 2进行**冒泡排序**

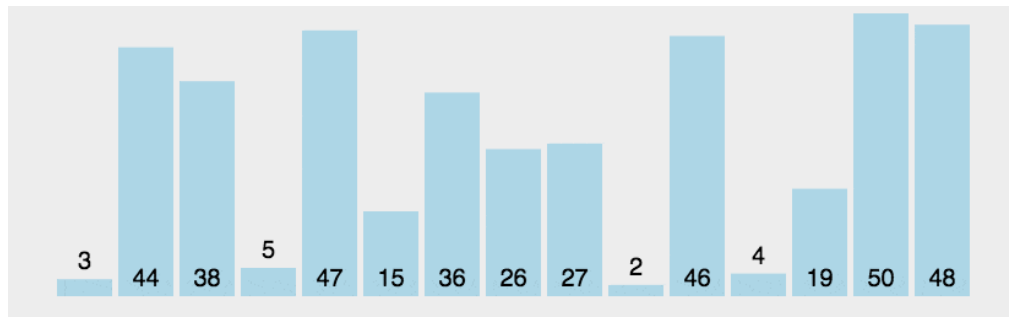
输出: 排序后的数组为 [1, 3, 4, 5, 6, 7, 8]

冒泡排序-Python

```
a = [8,6,7,5,1,3,4]
n = len(a)
for i in range(n):
    for j in range(0,n-i-1):
        if a[j] > a[j+1]:
            a[j],a[j+1] = a[j+1],a[j]
print("排序后的数组为",a)
```



流程控制语句



□ 让程序更加抽象

- 使用Python循环语句对8, 6, 7, 5, 1, 3, 4, 2进行冒泡排序

```
a = [8,6,7,5,1,3,4]
n = len(a)
for i in range(n):
    for j in range(0,n-i-1):
        if a[j] > a[j+1]:
            a[j],a[j+1] = a[j+1],a[j]
print("排序后的数组为",a)
```

3min 试一试

```
a = list(input("请输入待排序的数组: "))
def bubbleSort(a):
    n = len(a)
    for i in range(n):
        for j in range(0, n-i-1):
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]
bubbleSort(a)
print("排序后的数组:",a)
```

```
In [1]: runfile('')
```

请输入待排序的数组: 654321

排序后的数组: ['1', '2', '3', '4', '5', '6']



函数的定义与调用

□ 内置函数

- 标准函数，如幂运算用pow()

```
In [1]: 2**2  
Out[1]: 4
```

```
In [2]: pow(2,2)  
Out[2]: 4
```

→ 调用python中标准函数，提供实参2, 2

- 模块中的函数，如向下取整函数floor()

```
In [1]: import math
```

```
In [2]: math.floor(0.6)  
Out[2]: 0
```

```
In [3]: from math import floor
```

```
In [4]: floor(0.6)  
Out[4]: 0
```

→ 使用命令import导入模块

→ 使用模块中的函数

→ 无同名函数，可以使用import变种

→ 此时无需制定模块前缀



函数的定义与调用

□ 内置函数

- 使用help()获取有关函数的信息

```
In [1]: help(pow)
Help on built-in function pow in module builtins:

pow(base, exp, mod=None)
    Equivalent to base**exp with 2 arguments or
    base**exp % mod with 3 arguments

    Some types, such as ints, are able to use a more
    efficient algorithm when
    invoked using the three argument form.
```



函数的定义与调用

❑ 判断某个对象是否可以调用

- 使用内置函数callable()

```
In [1]: import math
```

```
In [2]: x = 0
```

```
In [3]: y = math.floor →
```

y此时可以看作内置函数，能够被调用

```
In [4]: callable(x)
```

```
Out[4]: False
```

```
In [5]: callable(y)
```

```
Out[5]: True
```

```
In [6]: y(0.7) →
```

```
Out[6]: 0
```

调用y



函数的定义与调用

□ 自定义函数

`def add_one(x):` 注意冒号
`return x + 1` 函数名称(形参) 函数
的返回值 赋值给变量 y

```
In [2]: x = 1
```

```
In [3]: y = add_one(x)
```

```
In [4]: y
```

```
Out[4]: 2
```

函数名称(实参)

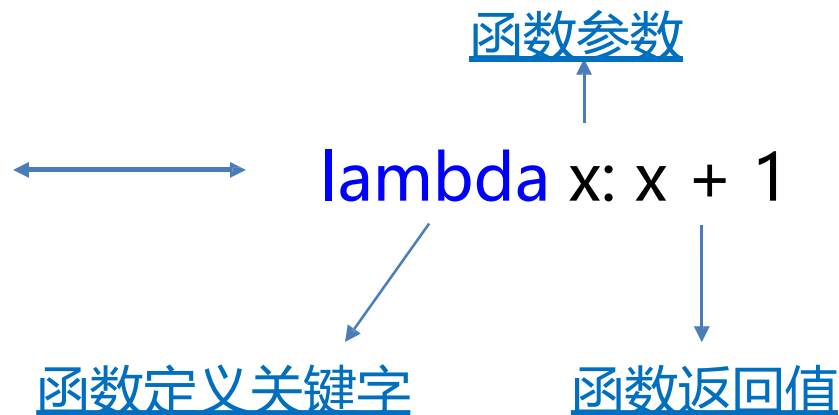


函数的定义与调用

□ 自定义函数

- 匿名函数

```
def add_one(x):  
    return x + 1
```



```
In [1]: (lambda x:x+1)(3)  
Out[1]: 4
```

- 没有函数名
- 只能在同一位置调用



函数的定义与调用

❑ 函数的参数能否修改

- 参数不可变，如数、元组、字符串

```
def try_to_change(x):
```

```
    x = 1
```

—————→ 函数内部给参数赋值

```
In [1]: x = 2
```

```
In [2]: runfile('')
```

```
In [3]: try_to_change(x)
```

```
In [4]: x
```

```
Out[4]: 2
```

—————→ 函数外部给参数赋值

—————→ 函数内部赋值对外部没有影响



函数的定义与调用

□ 函数的参数能否修改

- 参数可变，如列表

```
def try_to_change(x):
```

```
    x[0] = 1
```

→ 函数内部给参数赋值

```
In [1]: x = [3,1,5]
```

→ 外部赋值

```
In [2]: runfile('')
```

```
In [3]: try_to_change(x)
```

```
In [4]: x
```

```
Out[4]: [1, 1, 5]
```

→ 函数内部赋值对外部有影响



函数的定义与调用

❑ 函数的参数能否修改

- 变量关联的列表已被修改

`def try_to_change(x):` 不使用函数调用
`x[0] = 1`

```
In [1]: x = [3,1,5]
In [2]: y = x
In [3]: y[0] = 1
In [4]: x
Out[4]: [1, 1, 5]
```

- 避免两个变量指向同一列表，需创建列表的副本

创建整个列表的切片

```
In [1]: x = [3,1,5]
In [2]: z = x[:]

In [3]: x is z
Out[3]: False

In [4]: x == z
Out[4]: True

In [5]: z[0] = 1

In [6]: x
Out[6]: [3, 1, 5]
```

修改副本对原始列表无影响



函数的定义与调用

□ 函数的参数

- 多个参数

```
def add(x, y=1):  
    return x + 1
```

```
In [2]: x = 1
```

```
In [3]: y = add(x)
```

```
In [4]: z = add(y, 4)
```

```
In [5]: y
```

```
Out[5]: 2
```

```
In [6]: z
```

```
Out[6]: 3
```

```
In [7]: s = add(y, 4, 5)
```

→ `y = add(x=x, y=1)`

→ `y = add(x=y, y=4)`



函数的定义与调用

□ 函数的参数

- 多个参数

```
def name_age(name, age):  
    print(name, age)
```

```
In [2]: name_age('Mike', 18)  
Mike 18
```

```
In [3]: name_age(18, 'Mike')  
18 Mike
```

```
In [4]: name_age(age=18, name='Mike')  
Mike 18
```

```
In [5]: name_age(age=18, 'Mike')
```

- 不明确参数名时，按顺序赋值
- 明确参数名时，按参数名赋值
- 不明确参数名的参数必须在前



函数的定义与调用

□ 函数的参数

- 不定参数

1. 将args中的变量打包为一个元组来输出

```
def just_print(*args):  
    print(args)
```

2. *可以将元组中的变量再次拆解出来

```
def just_print2(*args):  
    print(*args)
```

```
In [2]: just_print(1,2,3)  
(1, 2, 3)
```

```
In [3]: just_print2(1,2,3)  
1 2 3
```



函数的定义与调用

□ 动手实践

15min 试一试

使用Python实现二分查找算法，并输出列表中所查找的元素的位置。

提示

- 二分查找：明确数字所处**位置的上下界**，每次查找都将区间分为两半
- 首先需要使用`sort()`对列表元素进行**排序**
- 如果上下界位置一致，说明都指向数字所处位置，就将数字返回；否则，找出区间的**中间元素**（上界和下界的平均值），再确定数字在左半部分还是右半部分，然后继续在数据所处部分进行查找



函数的定义与调用

□ 动手实践

试一试

使用Python实现二分查找算法，并输出列表中所查找的元素的位置

```
def search(sequence, number, lower=0, upper=None):
    sequence.sort()
    if upper is None:
        upper = len(sequence)-1
    if lower == upper:
        ← assert number == sequence[upper]
        return upper
    else:
        middle = (lower + upper)//2
        if number > sequence[middle]:
            return search(sequence, number, middle+1, upper)
        else:
            return search(sequence, number, lower, middle)
```

进行列表元素的排序 上下限
设为列表的首、末位置
判断数字与上下界元素的关系



类的定义与调用

□ 类的定义



类的定义与调用

□ 类的调用方法

```
class Train():  
    def __init__(self, color):  
        .....
```

参数将直接传入初始化函数

```
In [2]: train = Train(color = 'red')
```

→ 创建类的一个对象

```
In [3]: train.run()  
Wuwuwu~ I am running!
```

→ 调用类的成员函数

```
In [4]: print(train.color)  
red
```

→ 调用类的成员变量



类的定义与调用

□ 类的成员变量

```
class Train():  
    def __init__(self, color, model):  
        self.color = color  
        self.__model = model
```

→ 公开变量
→ 私有变量

↓
双下划线

```
In [2]: train = Train('red', 'CRH')
```

```
In [3]: print(train.color)  
red
```

```
In [4]: print(train.__model)
```

无法访问!



异常处理

□ try...except...finally语句

```
try:  
    a = 1/0  
except ZeroDivisionError as e:  
    a=1  
    print(e)  
finally:  
    print(a)
```

——> 此处e为ZeroDivisionError的别名

